

函数卡 · 遇到再查，别背

建 2026.09.24 补 2026.09.28 补齐路线后半段 (LLM API / Agent / RAG / 数据 / FastAPI, +47 张)

卡数 127 张 (127 张已实测) 怎么用 搜名字 → 看场景 → 复制代码

这套卡**不是拿来背的** —— 是写代码卡住时用来查的。每张卡上的代码和输出，都由 `tools/build_fn_cards.py` 真的跑过一遍再写进来，所以卡里不会出现"看着合理、其实报错"的示例。

用法 这套卡怎么用（30 秒）

这套卡覆盖你路线的哪几段

路线段	卡数	为什么要有它
① 数据库（手上这件事） SQL / sqlite3	52	第 3 课 + D8 作业；所有后端岗的默认底子
② 语言与文件 标准库 / 内置	26	json / os / re / datetime —— 写任何脚本都绕不开
③ 项目 v1：调大模型 LLM API	15	2026.10 立项后的第一块砖；换一家模型只改两行
④ 项目 v1：Agent Agent	10	Agent = LLM + 上下文 + 工具（看板 2.1.1 已开线）
⑤ 项目 v1：检索增强 RAG	7	你已经跑到 Recall@1 87% 的那条线
⑥ 数据与算法调用 数据	8	pandas / numpy —— 物联网线每天要用
⑦ 对外提供接口 FastAPI	7	你的 api.py；服务器 8009 端口跑的就是它
⑧ 按需查 第三方	2	requests / pandas 等的签名速查

● 这张表是为了防止上一版的毛病：第一版 79 张里数据库占 65%，而 LLM API / Agent / RAG 一张都没有 —— 原因是我拿「这一周正在上的课」当了锚。现在改成按你的整条路线分段，哪一段空着，一眼就看得出来。

四个阶段标记怎么读

标记	意思
现在	手上这件事直接要用 —— 第 3 课 + D8 作业。其余的先不用看。
近期	这一两周会撞到（排序、聚合、文件、JSON、日期）。撞到再回来查。
阶段A	2026.10–2027.01 做项目 v1 要用的：大模型调用 / Agent / RAG / pandas / FastAPI。现在不用会，但可以翻着看，知道"原来就这几下"。
以后	2027 起才用得上的（正则进阶、第三方库签名）。看到名字能想起来就行。
实测输出（绿块）	脚本跑出来的真实结果，不是手写的。红块 = 故意让它报错的现场，因为报错信息本身就是要认的东西。

🖨️ 大模型那些卡（LLM API / Agent / RAG）是怎么"实测"的：全部跑在你本机的 Ollama 上（`qwen2.5:3b` + `bge-m3`，`127.0.0.1:11434`），不联网、不花钱。卡上的问答是真调模型得到的那一次回答。⚠️ 但要说清楚：模型的回答每次会略有不同（`temperature=0` 也不保证逐字一样），所以你复跑时数字可能变 —— 这是模型的特性，不是卡写错了。

⚠️ 抄之前先改表名。SQL 那些卡跑在一份样板数据上（选修课三张表 `students / courses / enrollments` + 第 3 课的微信群三张表 `persons / groups / group_members`）—— 换成你自己的库，表名和列名都要跟着改。这正是第 3 课教的「换皮」。

内置函数不做成背诵卡（那条线已经关掉）—— 只留 8 张最常撞到的当查询用。要补哪些，说一声就加。

打印子集：只含 现在 —— 共 25 张

SQL 18 张

SELECT ... FROM ... 现在 SQL 所有查询的骨架：先说「要哪几列」，再说「从哪张表」 <pre>SELECT name, class FROM students;</pre> 实测输出 <table><thead><tr><th>name</th><th>class</th></tr></thead><tbody><tr><td>韩立</td><td>讯飞班</td></tr><tr><td>小明</td><td>讯飞班</td></tr><tr><td>小红</td><td>计科1班</td></tr><tr><td>小刚</td><td>计科1班</td></tr><tr><td>小美</td><td>计科1班</td></tr></tbody></table> （5 行） 📌 第3课① · 任何一条 SQL 都长这样 ⚠️ <code>SELECT *</code> 能跑但别常用 —— 列的顺序以后会变，代码会悄悄错位	name	class	韩立	讯飞班	小明	讯飞班	小红	计科1班	小刚	计科1班	小美	计科1班	WHERE 现在 SQL 先在脑子里选出全部，再一条条筛掉不要的 <pre>SELECT name, class FROM students WHERE class = '讯飞班';</pre> 实测输出 <table><thead><tr><th>name</th><th>class</th></tr></thead><tbody><tr><td>韩立</td><td>讯飞班</td></tr><tr><td>小明</td><td>讯飞班</td></tr></tbody></table> （2 行） 📌 第3课 · 筛选；D8 作业至少要用一次 ⚠️ 筛完只剩 2 行不是因为「表里只有 2 行」，是 WHERE 把它们筛掉了	name	class	韩立	讯飞班	小明	讯飞班
name	class																		
韩立	讯飞班																		
小明	讯飞班																		
小红	计科1班																		
小刚	计科1班																		
小美	计科1班																		
name	class																		
韩立	讯飞班																		
小明	讯飞班																		

ON / WHERE 分工 现在 SQL

ON = 怎么配对（横着拼），WHERE = 要哪些（竖着筛）

```
SELECT s.name, e.cid
FROM students s
JOIN enrollments e ON s.id = e.sid
WHERE e.cid = 1;
```

实测输出

name	cid
韩立	1
小明	1

(2 行)

第3课④·全课最该分清的一对

⚠ 把 `e.cid = 1` 挪到 ON 里，INNER JOIN 结果一样——但换成 LEFT JOIN 就天差地别（留在 ON 里 = 右表筛完再拼，放 WHERE 里 = 拼完再筛，会把 NULL 行筛掉）

COUNT(*) / COUNT(列) 现在 SQL

COUNT(*) 数行；COUNT(列) 只数**这一列不是 NULL**的行

```
SELECT COUNT(*) AS rows_all, COUNT(grade) AS
rows_graded FROM enrollments;
```

实测输出

rows_all	rows_graded
6	5

(1 行)

第3课⑥ 诊断第一步：单独数绳子表

⚠ 两者不一样就说明这列有 NULL——这个差值本身就是线索，不是 bug

JOIN ... ON（两表） 现在 SQL

拿一边的 id 去另一边配对，配对成功才出一行

```
SELECT s.name, e.cid
FROM students s
JOIN enrollments e ON s.id = e.sid;
```

实测输出

name	cid
韩立	1
韩立	2
小明	1
小明	3
小红	5
小刚	4

(6 行)

第3课④·本课核心

⚠ 结果行数 = 配对成功的次数，不是任何一张表的行数

三表 JOIN（中间那张是「绳子」） 现在 SQL

行数由中间那张绳子表说了算；两头都只是「换名字」

```
SELECT s.name AS student, c.name AS course,
e.grade
FROM students s
JOIN enrollments e ON s.id = e.sid
JOIN courses      c ON e.cid = c.id;
```

实测输出

student	course	grade
韩立	数据库	88.5
韩立	高数	76.0
小明	数据库	92.0
小明	机器学习	81.0
小红	英语	59.0
小刚	体育	NULL

(6 行)

第3课④·第6章换皮就是把它改名

⚠ 三张表的顺序可以换，但**每一句 ON 必须紧跟着它要拼的那张表**

坑：两边都有 id，不给表起别名 现在 SQL

两张表都有 `id` 列时，`ON students.id = courses.id` 里的 id 到底是谁的？

```
SELECT id, name FROM students JOIN courses ON
students.id = courses.id;
```

实测：真的会这样报错

OperationalError: ambiguous column name: id

报错现场——别名就是为了解决它

⚠ `ambiguous column name` 的正确解法是给表起别名然后写全：`s.id`、`c.id`

坑②：ON 接反 → 0 行（不报错） 现在 SQL

`groups.id` 是 1/2/3，`group_members.uid` 是 7/9/7——一个都对不上

```
SELECT persons.name AS person, groups.name AS
grp
FROM groups
JOIN group_members ON groups.id =
group_members.uid
JOIN persons      ON group_members.gid =
persons.id;
```

实测输出

person	grp
--------	-----

(0 行)

第3课④ 实测反例 1（低危：空的一眼看得出来）

⚠ JOIN 查出空表时的第一反应应该是「怀疑 ON」，而不是「数据没了」

坑②: id 重叠 → 照样出数据 (最危险) 现在 SQL

students.id 和 courses.id 都从 1 开始, 接反了照样配得上——于是给你一张看着完全合理的假表

```
SELECT s.name AS student, c.name AS course
FROM students s
JOIN enrollments e ON s.id = e.cid
JOIN courses      c ON e.sid = c.id;
```

实测输出

student	course
韩立	数据库
小明	数据库
韩立	高数
小红	高数
小美	机器学习
小刚	体育

(6 行)

📌 第3课④ 实测反例 2 (🔴 高危: 静默错误)

⚠️ 「不为空」**绝不等于**「对」。唯一的防线是**挑一行人工核对内容**——这张表里有**根本没选课的小美**、还有对不上课的小红, 一眼就能看出不对

证明它到底错在哪: 行数一样, 内容不同 现在 SQL

同一批数据, 正确写法和接反写法**都出 6 行**; 真正差的是其中 2 行的内容

```
SELECT
  (SELECT COUNT(*) FROM (
    SELECT s.name, c.name FROM students s
    JOIN enrollments e ON s.id = e.sid JOIN
    courses c ON e.cid = c.id
    EXCEPT
    SELECT s.name, c.name FROM students s
    JOIN enrollments e ON s.id = e.cid JOIN
    courses c ON e.sid = c.id
  )) AS missing_in_wrong,
  (SELECT COUNT(*) FROM (
    SELECT s.name, c.name FROM students s
    JOIN enrollments e ON s.id = e.cid JOIN
    courses c ON e.sid = c.id
    EXCEPT
    SELECT s.name, c.name FROM students s
    JOIN enrollments e ON s.id = e.sid JOIN
    courses c ON e.cid = c.id
  )) AS rows_that_are_fake;
```

实测输出

missing_in_wrong	rows_that_are_fake
2	2

(1 行)

📌 第3课④ 那张假数据卡的下半句 —— 用 SQL 自己证明

⚠️ 左边 = 正确结果里有、接反结果里没有的 (漏了 2 行); 右边 = 接反结果凭空多出来的 (假了 2 行)。⇒ 只数行数永远发现不了, 必须看内容

诊断三步: 先数绳子, 再数结果 现在 SQL

① 单独数绳子表 ② 数 JOIN 结果 ③ 两数一致不算过关, 再挑一行核对内容

```
SELECT (SELECT COUNT(*) FROM enrollments) AS
rope_rows,
      (SELECT COUNT(*) FROM students s JOIN
enrollments e ON s.id = e.sid) AS join_rows;
```

实测输出

rope_rows	join_rows
6	6

(1 行)

📌 第3课⑥ · 本课最实用的迁移技能

⚠️ 数字对不上 (空 / 多很多) → 几乎一定是 ON; 数字**对得上**也可能内容错 (id 重叠时)

LEFT JOIN 现在 SQL

左表一行都不能丢; 右表配不上就整行填 NULL

```
SELECT s.name, e.cid, e.grade
FROM students s LEFT JOIN enrollments e ON
s.id = e.sid;
```

实测输出

name	cid	grade
韩立	1	88.5
韩立	2	76.0
小明	1	92.0
小明	3	81.0
小红	5	59.0
小刚	4	NULL
小美	NULL	NULL

(7 行)

📌 第3课⑤ 换皮3 · 统计时最常见

⚠️ 把 LEFT 去掉, **没选课的人会整个消失** —— 人数从 5 变 4, 而且不报错

INNER vs LEFT: 差在哪一行 现在 SQL

同一条查询, 只差 LEFT 四个字母, 结果少一行

```
SELECT s.name FROM students s JOIN enrollments
e ON s.id = e.sid;
```

实测输出

```
name
----
韩立
韩立
小明
小明
小红
小刚
(6 行)
```

📌 对照上一张卡看: 5 个人变成了 4 个人, 小美没了

⚠️ 要「一个都不漏」的清单 (点名、对账、覆盖率) 时必须 LEFT

看这个库有哪些表 现在 SQL

不用装数据库工具, 一条 SQL 就能看清家底

```
SELECT name FROM sqlite_master WHERE type =
'table' ORDER BY name;
```

实测输出

```
name
-----
courses
enrollments
group_members
groups
persons
students
(6 行)
```

📌 第3课①「这库里一共几张表」那道题的正确做法

⚠️ sqlite_master 是 SQLite 专有; MySQL 里叫 SHOW TABLES

看一张表的结构 现在 SQL

列名、类型、是否主键, 一条 PRAGMA 全给你

```
PRAGMA table_info(enrollments);
```

实测输出

```
cid | name | type | notnull | dflt_value |
pk
-----+-----+-----+-----+-----
+---
0 | sid | INTEGER | 0 | NULL |
0
1 | cid | INTEGER | 0 | NULL |
0
2 | grade | REAL | 0 | NULL |
0
(3 行)
```

📌 忘了列名时先看这个, 别猜

⚠️ PRAGMA 不是标准 SQL (且不吃 ? 占位符) —— 表名只能拼字符串, 只用于自己写死的表名

CREATE TABLE 现在 SQL

并列三件事: 列定义、主键、外键约束

```
CREATE TABLE enrollments_new (
    sid INTEGER NOT NULL REFERENCES
students(id),
    cid INTEGER NOT NULL REFERENCES
courses(id),
    grade REAL,
    PRIMARY KEY (sid, cid)
);
```

实测输出

```
cid | name | type | notnull | dflt_value |
pk
-----+-----+-----+-----+-----
+---
0 | sid | INTEGER | 1 | NULL |
1
1 | cid | INTEGER | 1 | NULL |
2
2 | grade | REAL | 0 | NULL |
0
(3 行)
```

📌 第3课③ · D8 作业的第一件事

⚠️ **复合主键** PRIMARY KEY (sid, cid) 直接防住「同一个人重复选同一门课」—— 比在 Python 里查一遍再插可靠

INSERT INTO 现在 SQL

列名写全，值用 ? 占位 (Python 里)

```
INSERT INTO students (id, name, class) VALUES
(6, '小七', '计科1班');
```

实测输出

```
id | name | class
---+-----+-----
6  | 小七 | 计科1班
(1 行)
```

📌 D8 作业：插 3 行数据

⚠️ 列名和值数量必须一致，否则报 `table has N columns but M values were supplied`

REFERENCES 外键：写了 ≠ 生效 现在 SQL

`sid INTEGER REFERENCES students(id)` 只是声明。
SQLite 默认**不检查它**——一个不存在的学生照样能选上课，而且**不报错**

```
import sqlite3, tempfile, os, shutil
d = tempfile.mkdtemp(); p = os.path.join(d, "t.db")
con = sqlite3.connect(p, isolation_level=None)
# 自动提交：避开"事务里设 pragma 会被忽略"那个坑
for ddl in ["CREATE TABLE students (id INTEGER
PRIMARY KEY, name TEXT)",
            "INSERT INTO students VALUES (1,'韩立')",
            "CREATE TABLE courses (id INTEGER
PRIMARY KEY, name TEXT)",
            "INSERT INTO courses VALUES (1,'数据库')",
            "CREATE TABLE enrollments (sid
INTEGER NOT NULL REFERENCES students(id),
            " cid INTEGER NOT NULL REFERENCES
courses(id), PRIMARY KEY (sid, cid))"]:
    con.execute(ddl)

print("① 声明被记下来了么 =", [(r[3], r[2], r[4])
for r in con.execute("PRAGMA
foreign_key_list(enrollments)"))]
print("② 默认开关 =", con.execute("PRAGMA
foreign_keys").fetchone()[0], "(0 = 关)")
con.execute("INSERT INTO enrollments VALUES
(999, 1)")
print("③ 插一个不存在的学生 999 → 进得去，库里 =",
con.execute("SELECT * FROM
enrollments").fetchall())

con.execute("DELETE FROM enrollments");
con.execute("PRAGMA foreign_keys = ON")
print("④ 打开开关后再插同一条 →", end=" ")
try:
    con.execute("INSERT INTO enrollments
VALUES (999, 1)")
except Exception as e:
    print(type(e).__name__, ":", e)
con.close()
c2 = sqlite3.connect(p, isolation_level=None)
print("⑤ 新连接又回到 =", c2.execute("PRAGMA
foreign_keys").fetchone()[0], "(每条连接各自，不
写进文件)")
c2.close(); shutil.rmtree(d,
ignore_errors=True)
```

实测输出

```
① 声明被记下来了么 = [( 'cid', 'courses', 'id'),
( 'sid', 'students', 'id')]
② 默认开关 = 0 (0 = 关)
③ 插一个不存在的学生 999 → 进得去，库里 = [(999,
1)]
④ 打开开关后再插同一条 → IntegrityError : FOREIGN
KEY constraint failed
⑤ 新连接又回到 = 0 (每条连接各自，不写进文件)
```

📌 D8 建表直接用；你 `prescription_db.py` 里写的 FOREIGN KEY 也是同一条规则

⚠️ ⚠️ `PRAGMA foreign_keys = ON` 必须每次连上就设一遍
—— 它不写进文件、新连接回到 0。🔴 更阴的一条：在未提交的

事务里设它会被静默忽略（读回来还是 0，不报错也不警告）→
要么 `isolation_level=None`（自动提交），要么先 `commit()` 再设。已经混进去的脏数据可以用 `PRAGMA foreign_key_check` 事后揪出来。另：同一条 `CREATE TABLE` 里的 `PRIMARY KEY (sid, cid)` 默认就生效 —— 一行管用一行不管

六行骨架：连接 → 游标 → 执行 → 取 → 关 **现在**

sqlite3

连 Python 和数据库的全部动作就这六行

```
import sqlite3
conn = sqlite3.connect(":memory:") #
# 文件路径: :memory: = 内存库
cur = conn.cursor()
cur.execute("CREATE TABLE t (x)")
cur.execute("INSERT INTO t VALUES (1), (2)")
conn.commit() #
# 写操作必须 commit
for row in cur.execute("SELECT x FROM t"):
    print(row)
conn.close()
```

实测输出

```
(1,)
(2,)
```

📌 第3课 · 所有 SQL 作业的外壳

⚠️ `commit()` 忘写的后果见下一张卡 —— 它和 `close()` 是两件事

commit() vs close(): 谁在丢数据 **现在** sqlite3

P-014 的实测：换一个连接去读，看数据还在不在

```
import sqlite3, tempfile, os, shutil
d = tempfile.mkdtemp(); p = os.path.join(d, "k.db")

c = sqlite3.connect(p); c.execute("CREATE TABLE t(x)")
c.execute("INSERT INTO t VALUES (1)")
c.close() #
# 只 close, 不 commit
c2 = sqlite3.connect(p)
print("commit X / close ✓ → 另一个连接读到",
c2.execute("SELECT COUNT(*) FROM t").fetchone()[0], "行")
c2.close()
```

```
c = sqlite3.connect(p); c.execute("INSERT INTO t VALUES (2)")
c.commit(); c.close() #
# 先 commit, 再 close
c2 = sqlite3.connect(p)
print("commit ✓ / close ✓ → 另一个连接读到",
c2.execute("SELECT COUNT(*) FROM t").fetchone()[0], "行")
c2.close(); shutil.rmtree(d, ignore_errors=True)
```

实测输出

```
commit X / close ✓ → 另一个连接读到 0 行
commit ✓ / close ✓ → 另一个连接读到 1 行
```

📌 第3课① 检索题（当场答错的那道）· P-014

⚠️ 丢数据的凶手是 `commit`，不是 `close`。`close()` 忘写的代价是占资源（见下张卡），不是丢数据

close() 忘了写会怎样 现在 sqlite3

连接不关 → 文件被锁住 → 连删都删不掉

```
import sqlite3, tempfile, os, shutil
d = tempfile.mkdtemp(); p = os.path.join(d, "k.db")
c = sqlite3.connect(p); c.execute("CREATE TABLE t(x)"); c.commit()
try:
    os.remove(p) #
    连接还开着, Windows 不让删
    print("删掉了 (没锁)")
except PermissionError as e:
    print("PermissionError:", e)
c.close()
os.remove(p) #
    关掉之后再删就成功了
print("close 之后再删: 成功")
shutil.rmtree(d, ignore_errors=True)
```

实测输出

```
PermissionError: [WinError 32] 另一个程序正在使用
此文件, 进程无法访问。:
'E:\\Temp\\tmpmik2j1z7\\k.db'
close 之后再删: 成功
```

🔴 P-014 的另一半: close 的真实代价

⚠️ Linux/Mac 上删得掉, 只有 Windows 会当场报 WinError 32 —— 所以这个坑在 Windows 上反而更容易被发现

fetchone / fetchall / 直接遍历 现在 sqlite3

三种取结果的方式, 选中一条就不可能再回头

```
import sqlite3
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE t(x)")
conn.executemany("INSERT INTO t VALUES (?)",
[[1,], (2,), (3,)])
cur = conn.cursor()
cur.execute("SELECT x FROM t")
print("fetchone →", cur.fetchone()) #
    拿走第 1 行, 游标前移
print("fetchall →", cur.fetchall()) #
    剩下的全拿走
cur.execute("SELECT x FROM t")
print("遍历 →", [r[0] for r in cur])
conn.close()
```

实测输出

```
fetchone → (1,)
fetchall → [(2,), (3,)]
遍历 → [1, 2, 3]
```

🔴 取查询结果

⚠️ 游标是**一次性的**: fetchone() 之后再 fetchall() 拿不到已经取走的那行。要重复用就把结果存进 list

executemany() 批量插入 现在 sqlite3

一次插 20 行, 别写 20 条 execute

```
import sqlite3
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE s(name TEXT, score REAL)")
rows = [("小明", 88.5), ("小红", 92.0), ("小刚", 76.5)]
conn.executemany("INSERT INTO s VALUES (?, ?)", rows)
conn.commit()
print(conn.execute("SELECT COUNT(*) FROM s").fetchone()[0], "行")
for r in conn.execute("SELECT * FROM s ORDER BY score DESC"):
    print(r)
conn.close()
```

实测输出

```
3 行
('小红', 92.0)
('小明', 88.5)
('小刚', 76.5)
```

🔴 灌测试数据 / 批量写爬下来的数据

⚠️ 参数是一个可迭代对象, 每项对应一条语句; 写成 executemany(sql, (1,2)) 会把它当成两行单列

`?` 占位符 (防注入) 现在 sqlite3

值永远走参数, 不要用 f-string 拼进 SQL

```
import sqlite3
conn = sqlite3.connect(":memory:")
conn.execute("CREATE TABLE t(name TEXT)");
conn.execute("INSERT INTO t VALUES ('小明')")
who = "小明" OR '1'='1' #
    想注入的人会这么输
n = conn.execute("SELECT COUNT(*) FROM t WHERE name = ?", (who,)).fetchone()[0]
print("参数化 (对) →", n, "行")
bad = "SELECT COUNT(*) FROM t WHERE name = '%s'" % who
n2 = conn.execute(bad).fetchone()[0]
print("拼字符串 (错) →", n2, "行")
conn.close()
```

实测输出

```
参数化 (对) → 0 行
拼字符串 (错) → 1 行
```

🔴 所有带用户输入的查询

⚠️ **单元素也必须写成** (who,) —— 少了逗号 (who) 只是个字符串, 会被当成一个可迭代对象逐字符拆开, 报 Incorrect number of bindings

row_factory: 按列名取值 现在 sqlite3

设一次 row_factory，之后 row['name'] 就能用

```
import sqlite3
conn = sqlite3.connect(":memory:")
conn.row_factory = sqlite3.Row      # ← 就这一行
conn.execute("CREATE TABLE s(id INTEGER, name TEXT)")
conn.execute("INSERT INTO s VALUES (1, '小明')")
row = conn.execute("SELECT id, name FROM s").fetchone()
print("按下标 →", row[0], row[1])
print("按列名 →", row['id'], row['name'])
print("变字典 →", dict(row))
conn.close()
```

实测输出

```
按下标 → 1 小明
按列名 → 1 小明
变字典 → {'id': 1, 'name': '小明'}
```

📌 结果列一多，下标就记不住了

⚠️ row_factory 设在 **connection** 上，不是 cursor 上（设 cursor 上是另一套写法）

函数卡 v1 · 2026.09.24 | 世界：米黄方格纸 · 实验记录本

数据 _函数卡数据.js（由 tools/build_fn_cards.py 实测生成，别手改） | 主题 _主题.css | 重跑： python tools/build_fn_cards.py